Security self-assessment report

International Track deliverable 05: **permission-tier implementation and known-risk list.** StudyRoute is a sandbox with synthetic data; this report says what is enforced, where, how it is tested, and what is *not* solved. The control-by-control threat matrix is in SECURITY.md; the design is in TECHNICAL_DOCUMENTATION.md. The attack tools were re-run on the final code on 18 October 2026, including against the bank review desk (146 of 146 attacks defended, 48 of 48 browser checks); the raw output is in `submission/evidence/security_2026-10-18/` and the reading of it in SECURITY_TESTING.md. They were written by the team that wrote the app; no outside penetration test has been done.

1. Permission tiers

Authorisation is enforced in the **domain layer** (`backend/app/domain/rbac.py`), not only in the UI or the route. A call needs both the role's permission **and** the matching scope in the access token, so a narrowed token cannot do more than it was issued for. Objects belong to a household: a request for another household's object gets `404` , the same answer as for a missing one.

1.1 Tiers

Tier	Who	What	How it is enforced
0. Public	Anyone	Sign in, refresh, sign out, `GET /health` , `GET /sandbox` (sandbox only)	Rate limit by IP. `/sandbox` returns 404 in production
0b. Provider	A provider adapter	`POST /webhooks/provider`	HMAC signature with timestamp window and event-id de-duplication, not a bearer token
1. Read	Parent, student, admin, AI (read scopes)	Quotes, payments, recipients, rules, checklist, planner	Role permission plus token scope plus household ownership
2. Prepare	Parent, student (limited), AI (draft and request only)	Create quotes, drafts, recipients, documents; request approval	Role permission plus scope. A student cannot draft; the AI can only propose
3. Decide (step-up)	**Human parent only**	Approve a payment, allowlist or de-list a recipient	Everything in tier 2, plus a **fresh MFA proof** (5 minutes, single use, bound to the user, the action and the object), plus the approval summary hash
4. Administer	Admin (reviewer)	Change rules (audited), verify the audit chain, review KYC, reject a payment, drive the simulated provider (sandbox only)	Admin-only permission. An admin **cannot** approve, draft or quote
Never	The AI service account	Approve, submit, cancel, reject, allowlist, change limits, edit config, verify the audit log	Not in its permission set or token scopes; the approve path also returns `AI_CANNOT_APPROVE` and writes an audit event

1.2 Permission matrix (generated from the code)

`yes` means the role holds the permission. A test (`test_security_self_assessment.py`) fails if this table and `ROLE_PERMISSIONS` ever differ.

Permission	Parent	Student	Admin	AI agent
`quotes:create`	yes	yes	-	yes
`quotes:read`	yes	yes	yes	yes
`payments:read`	yes	yes	yes	yes
`payments:draft`	yes	-	-	yes
`payments:request_approval`	yes	-	-	yes
`payments:approve`	yes	-	-	-
`payments:cancel`	yes	-	-	-
`payments:reject`	yes	-	yes	-
`recipients:create`	yes	yes	-	-
`recipients:read`	yes	yes	yes	yes
`recipients:allowlist`	yes	-	-	-
`config:read`	yes	yes	yes	yes
`config:write`	-	-	yes	-
`audit:read`	yes	-	yes	-
`audit:verify`	-	-	yes	-
`kyc:read`	yes	yes	yes	yes
`kyc:submit`	yes	yes	-	-
`kyc:review`	-	-	yes	-
`planner:read`	yes	yes	-	yes
`checklist:read`	yes	yes	yes	yes
`chat:use`	yes	yes	-	-
`mfa:issue`	yes	-	yes	-

Permission	Parent	Student	Admin	AI agent
`documents:read`	yes	yes	-	yes
`documents:write`	yes	yes	-	-
`data:reset`	yes	-	-	-
`sandbox:control`	-	-	yes	-
`bank:read`	yes	yes	yes	-
`bank:transfer`	yes	-	-	-
`escrow:read`	yes	yes	yes	-
`escrow:write`	yes	-	-	-
`escrow:freeze`	-	-	yes	-
`chain:read`	yes	yes	yes	-
`chain:anchor`	-	-	yes	-
`review:read`	yes	yes	yes	yes
`review:answer`	yes	-	-	-
`review:decide`	-	-	yes	-

1.3 Guards that apply on top of permissions

Action	Extra guards (all must pass)
Request approval	Quote not expired, KYC verified, recipient allowlisted, mock sanctions clear, per-transaction and daily caps
Approve	The same guards again, fresh MFA proof for this payment, `confirm_summary_hash` equals the frozen summary, `Idempotency-Key`, optimistic lock
Allowlist a recipient	Fresh MFA proof for this recipient; renaming a recipient resets its allowlist
Webhook	Valid HMAC, timestamp within 300 seconds, unseen event id, matching provider and UETR
Change a rule	Admin permission, a reason, audit event with before and after hashes

Each row has a test; see `backend/tests/test_api_payment_guards.py`, `test_api_roles.py`, `test_api_ai_rbac.py`, `test_api_webhooks.py` and `test_rbac.py`.

2. Known-risk list

Severity is the team's own judgement for a **sandbox that may one day be taken towards production**. "Accepted" means we chose not to fix it in this submission and say so openly.

ID	Risk	Severity	Mitigation in place	Residual status
R1	**Sandbox MFA code is a fixed documented value**, and demo credentials are shown on the sign-in page	Medium (sandbox only)	Both exist only in sandbox mode and `SHOW_DEMO_HINTS` can switch the hints off; production forces the fixed code to `None` and refuses to start without real secrets	Accepted for the demo. Real TOTP works in every mode
R2	**No external identity provider or PKCE flow**: password plus our own tokens	Medium	Argon2id, ES256 tokens (15 minutes), rotation with reuse detection, lockout, MFA step-up	Accepted; documented as a simplification
R3	**Updated: browser session is HttpOnly, not JS storage**; XSS can still act as the signed-in user	Medium	SameSite=Strict, exact Origin + custom-header CSRF checks, Secure production cookie, server revocation, memory-only bound access JWT; CSP and output escaping remain	Mitigated, not eliminated. `test_browser_session.py`; legacy bearer refresh rotation unchanged
R4	**Plain HTTP in the compose stack**	Medium for production, none for the sandbox	HSTS and other headers are sent; TLS termination is expected in front of nginx	Open until deployed behind TLS
R5	**The API trusts `X-Forwarded-For`** (`--forwarded-allow-ips="*"`) so per-IP rate limits see the real client	Medium if the API port were exposed	Compose publishes only nginx, which overwrites the header	Accepted; do not publish the API port
R6	**Rate-limit counters live in process memory**	Low	Correct for one process; limits would multiply behind several workers	Open: use a shared store before scaling out
R7	**Audit log is tamper-evident, not tamper-proof.** Someone with database file access can drop the triggers (the demo script does exactly this)	Medium	Verify detects inconsistent edits, not a coordinated rewrite/rehash by the database owner. A separately protected trusted-prefix checkpoint can expose that rewrite or truncation; see `AUDIT_TRUST_BOUNDARY.md`	Open: independent checkpoint retention/publication is manual
R8	**SQLite has no database roles**, so "application cannot update audit rows" is enforced by triggers only	Low	Triggers block `UPDATE` and `DELETE`; test `test_application_role_cannot_update_or_delete_audit_rows`	Open: on Postgres, also revoke the privileges
R9	**Prompt-injection filter is heuristic.** A new phrasing might slip past the policy patterns	Medium	Defence in depth: no tool can approve, submit, cancel or change limits; the number guard blocks invented figures; the model sees masked data only; a person must approve with MFA	Accepted: the worst outcome is a harmless reply, not an action
R10	**The number guard checks numbers, not wording.** A model could state a non-numeric claim such as "this is allowed"	Medium	The system prompt forbids it; questions about legality get a fixed "I cannot say whether something is legal" answer and the "Indicative. Confirm with your AD bank." line; template mode needs no model	Accepted; human reviews the engine's approval summary

ID	Risk	Severity	Mitigation in place	Residual status
R11	**Most regulatory values are unverified** (living-expense limit, cut-off, calendars, checklists)	High if presented as real	Each is in config with `verified: false`, a yellow **Not verified** badge and an indicative disclaimer; listed in `docs/OPEN_QUESTIONS.md`. On 18 Oct 2026 four study-abroad rules (one AD branch, one academic year at a time, release per the institution's estimate, international cards for education) were read in the official FEPD-1 Circular No. 40 and marked verified with their section numbers; the USD 2,000 a month living figure is **not** in that text and stays unverified	Partly closed; the rest is open until checked against official sources. Nobody outside the team has reviewed the reading
R12	**Sanctions screening is a test token**, not a real list	Low (sandbox)	Labelled "mock" everywhere; blocked names are audited	Accepted
R13	**Sandbox secrets are generated under `data/`** (gitignored); the supplied larger workspace ZIP also contained credentials	High until owner-side revocation	The repaired release excludes them and scanning rejects credential files. This does not establish the safety of prior ZIPs or Git history; production needs independently managed secrets	Open: revoke exposed provider credentials; rotate deployed signing/webhook secrets carefully, migrate encryption keys without data loss
R14	**Dependencies could carry known vulnerabilities**	Medium	Backend versions are pinned in `requirements.txt`; the npm lock file pins the web UI. A `dependency-audit` CI job runs `pip-audit` and `npm audit --omit=dev` on every push. On 14 Oct 2026 both reported no known vulnerabilities	Mitigated. An audit only knows published advisories, so it must keep running
R15	**No cloud or live-model path has run against a real provider.** The Anthropic and OpenAI-compatible clients, Gemini, Groq and the Hugging Face Space adapter were exercised only against fakes and mocked HTTP (`docs/ASSISTANT_CLOUD_VERIFICATION.json` says `live_cloud: false`)	Low	`check_ai` verifies a configured model end to end before use; any failure, or an invented number, is reported or falls back instead of acting; the assistant cannot approve or move money in any mode	Open until a real key is used and the result is recorded
R16	**No CAPTCHA** on sign-in	Low	Rate limit by IP and account lockout	Accepted (optional in the design)
R17	**Simulated systems could be mistaken for real ones**	Medium	Banner on every screen, `X-Sandbox: true` on every API response, "simulated" on every provider-related view, no claim of a live bank or FX feed	Mitigated
R18	**The assistant can now create a quote** (a state change) when it plans a payment from a sentence	Low	The RBAC matrix already allows the AI to create quotes; it still cannot create a payment, approve, submit, cancel or allowlist. A message flagged by the policy check creates nothing; each quote is owner-scoped, audited as `quote.created` by the AI actor, and limited by the 20-chats-a-minute rate limit. Tests: `test_ai_tasks.py`	Accepted: a stray quote is harmless and expires after five minutes
R19	**The bank network is a simulation with invented rails, fees and limits**, so its numbers could be mistaken for a real scheme's rules	Medium if presented as real	Labelled simulated on the screen and in every response, banks all start with `EXMP`, nothing is marked verified, and the parameters live in `config/bank_network.yaml` and `docs/OPEN_QUESTIONS.md`	Mitigated. Replace with verified scheme rules before any real use

ID	Risk	Severity	Mitigation in place	Residual status
R20	**A failed bank transfer is unwound exactly**, with no return fee and no partial return, which is kinder than a real bank	Low	The behaviour is documented in `docs/BANK_NETWORK.md`; the unwind is a reversing journal, so the books still balance and the audit log records the return	Accepted for the demo
R21	**The parent can pay from any account of the demo household**, including the student's UK account, and bank transfers have no daily total	Low (sandbox)	Only a parent with a fresh MFA proof for that transfer can confirm; the per-transaction cap, rail limits and a 20-a-minute rate limit apply; every step is audited	Accepted; a real bank would model mandates and daily limits
R22	**A rewritten audit log** can still pass the SHA-256 hash chain if an attacker recomputes every hash after the edit	High without an outside check	The reviewer anchors the audit head hash on the sandbox chain (`AuditAnchor` contract). `GET /chain/verify` compares the database with every anchor and reports `MISMATCH` or `MISSING_EVENT`. A test performs exactly this attack and shows the chain check passing while the anchor check fails	Open against database-owner attacks: both logs share SQLite; use an independently protected checkpoint (docs/AUDIT_TRUST_BOUNDARY.md)
R23	**The escrow's chain history could be edited** to hide a payment (the in-memory chain is rebuilt from the database)	Medium	`chain_log` is append-only (database triggers) and hash-chained: each row covers the one before it. On every start the log is replayed and each row's hash is recomputed; a changed or missing row stops the chain (`LOG_CORRUPT`) and writes are refused	Open against a database owner who drops triggers and recomputes both hash chains; replay is not an independent trust root
R24	**A stolen parent session could lock or move escrow funds**	Medium	Locking funds, paying a tranche early and cancelling each need a fresh MFA proof bound to that exact escrow and used once, plus an Idempotency-Key; the per-transaction cap and KYC gate apply; every step is audited. The AI has no escrow permission	Mitigated
R25	**A reviewer could abuse the freeze**	Low	The `TuitionEscrow` contract gives the compliance role only `freeze` and `unfreeze`: no function lets it release, refund or cancel, which a contract test checks. Every freeze needs a written reason and is audited	Mitigated by design
R26	**The simulated chain could be mistaken for a real blockchain**	Medium if presented as real	Every response says `simulated`, the screen says "no network, no real tokens, not money", and the whole surface answers 404 outside the sandbox. It is an in-memory EVM (py-evm), not a public or consortium chain	Accepted; deploying to a real EVM chain is a documented next step
R27	**An EVM library (py-evm, with its dependencies) and a build tool (solc) enter the supply chain**	Low	All are pinned and permissively licensed (MIT, BSD or Apache-2.0); every dependency ships prebuilt wheels for Linux and Windows, so no compiler is needed on the server; the backend runs committed artifacts, so `solc` is never installed there; a test checks every artifact against its source file by SHA-256; `pip-audit` and `npm audit` on `contracts/` are clean	Accepted
R28	**E-mail addresses could be discovered by trying to lock them.** A real account answered 429 after repeated failures while an unknown address kept answering 401 (found by the security probe, 15 Oct 2026)	Medium	Fixed: `FailedLoginTracker` counts failed sign-ins per e-mail whether or not the account exists, so both lock after the same number of failures with the identical 429 answer and `Retry-After`. The count is bounded in memory. Regression tests cover real versus made-up addresses	Fixed

ID	Risk	Severity	Mitigation in place	Residual status
R29	**The sandbox shows its demo password and MFA code** on the sign-in page and in `GET /health` so judges need no set-up	Medium if a sandbox is ever exposed to the internet	Only while `APP_ENV=sandbox`; production returns no hints (tested). All data is synthetic. To expose a sandbox anyway, set `SHOW_DEMO_HINTS=false`, `SEED_DEMO_PASSWORD` and `SANDBOX_MFA_CODE`	Accepted for the demo; never publish a sandbox
R30	**The container trusts `X-Forwarded-For`** (`--forwarded-allow-ips=*`) because only nginx can reach the API. If someone published the API port, a client could forge its address and dodge the per-IP rate limit	Low	Compose does not publish the API port (a test enforces it); nginx overwrites the header; account lockout still stops password guessing from any address. The application itself ignores the header unless the connecting peer is a trusted proxy	Accepted; documented
R31	**The Next.js dev server (`npm run dev`) is not hardened**	Low	For development only and bound to loopback. The security headers and CSP come from nginx (`docker compose up`); the browser audit tests the production build under that CSP	Accepted
R32	**The trained reader can misread a message or be fooled by new wording** (it is a statistical model, not a rule)	Medium	It only proposes. Values come from deterministic code that refuses what it cannot read exactly; if the rule reader and the model read a field differently the field is left empty and the assistant asks; starting something needs more confidence than explaining; an attack flag can only add a block. It has no permission of its own, so the AI account still cannot approve, submit, cancel or change a limit, and a human confirms every draft with MFA. Measured on data it never trained on in `docs/AI_EVALUATION.md` (blind set: attacks blocked 92.9%, 2.0% of harmless messages blocked)	Accepted: some novel attacks and some harmless questions are misrouted; the consequence is a refusal or a quote card, never a payment
R33	**The model file could be swapped or tampered with** (supply chain)	Medium	The model is an ONNX graph (data, not a pickle), so loading it cannot run code. Its SHA-256 and the vocabulary's are checked against `manifest.json` at start-up and the reader is not used on a mismatch (tested). The manifest records the hash of the exact training data (`test_ml_dataset.py`) and a quality floor test (`test_nlu_quality.py`) fails a worse model. The base model is MIT-licensed and downloaded only at training time	Accepted: someone who can edit both the model and its manifest in the repository can still swap it; code review and repository access control are the answer, and signing the manifest is a next step
R34	**The training data is synthetic and English-centred** (generated messages plus a UK bank's public questions), so the model may behave differently on real parents' messages, and its thresholds were tuned on generated data	Medium	Held-out templates, real customer questions and a blind hand-written set are reported separately and honestly; the rules remain as the fallback and as a second reader; no real person's data is used. `docs/AI_AGENT.md` lists the limits	Accepted until it is tested with real users
R35	**Over-trust**: a reader may assume the model decides money matters	Low	The chat shows "How I read this" (what the model marked and what plain code made of it), the audit log records the reader's id, hash and confidence on every AI event, and the engine still computes every number	Mitigated

ID	Risk	Severity	Mitigation in place	Residual status
R36	**Data and model licences**: the training data includes Banking77 (CC BY 4.0) and the base model is `bge-small-en-v1.5` (MIT)	Low	Both are credited in `docs/THIRD_PARTY.md`; only short questions are used; the fine-tuned weights are the team's own derivative	Mitigated
R37	**The reader sees the message unmasked** (it runs inside the API)	Low	It runs locally on CPU with no network. Personal data is still masked before any text reaches an external language model (unchanged), and the audit log stores only a hash of the message	Mitigated
R38	**A dependency can phone home.** ONNX Runtime 1.31 sends Microsoft telemetry on Linux by default (found by a packet capture of the real container, 15 Oct 2026)	Medium	Switched off in code and in the image, with tests; the capture was repeated and showed no packet to a public address. The backend does not need the internet (only an optional language-model provider does), so an operator can also block its outbound traffic	Mitigated for this dependency. Other libraries were not each audited for egress beyond that capture
R39	**The dialogue policy was trained in a simulation**, against simulated parents and a reward the team chose, so it may behave badly with real people (a question too many, or one too few)	Medium	Its steps are limited by plain-code rules (it can never plan with the amount missing or read two ways, and cannot confirm twice) and none of them moves money. The reward and the simulator's assumptions are written down, and the evaluation retrains it under different rewards to show how much the behaviour depends on them (`docs/AI_RL_EVALUATION.md`). The person can always say no or correct a value. `DIALOGUE_MODE=off` restores plan-as-soon-as-known	Accepted until it is tried with real users
R40	**The policy file could be tampered with**	Low	JSON data, not code, checked against its manifest's SHA-256 at start-up and refused on any mismatch or wrong shape (tested); even a hostile file can only rank actions the rules allow (tested with a policy that always wants to plan)	Mitigated
R41	**The deadline choice and the currency-drop stress test could be read as predictions.** They rest on synthetic settlement times and synthetic rates, so a person might rely on "arrives in time" or on the stress figures as if they were forecasts	Medium	Both are computed by the fee engine from the quote (no separate maths; tested to reproduce the stored baseline and to move only the rate-driven part), labelled "Indicative. Confirm with your AD bank." and, for the stress test, "a what-if on synthetic rates, not a forecast"; the assistant only repeats engine figures (number guard) and cannot approve; the person still chooses the provider and approves with a fresh MFA code. Settlement times are configuration with a verified flag	Accepted: real providers' times and rates are unknown
R42	**The content-security policy allows a few hashed inline scripts, and four libraries are patched when installed.** The static export of the UI contains small inline bootstrap scripts, so the policy lists their SHA-256 hashes; `sonner`, `react-style-singleton`, `input-otp` and Radix Select are changed by `web/scripts/postinstall.mjs` so that none of them adds an inline `<style>`. A hash list that grew too wide, or a patch that drifted from the library, would weaken or break the policy	Low	The hashes are computed by the build from the exported pages (`web/scripts/csp-hashes.mjs`), never written by hand, and the policy has no `'unsafe-inline'` anywhere; the patch script fails the install loudly when a library's code changes; the Playwright smoke test and `browser_audit.mjs` run the production build under the real policy and fail on any violation, and the audit confirms that an injected inline script and style are still blocked	Accepted: the alternative is a weaker policy

ID	Risk	Severity	Mitigation in place	Residual status
R43	**The student cost figures and arrival steps could be taken for real, current advice.** The cost-of-living figures for 20 cities are invented, and the arrival steps are general; a person might budget a move or skip a visa step because of them	Medium	Every figure is in config with `verified: false`, a source to replace and the badge "Not verified"; every answer and screen says "invented for the sandbox, not a survey", "compare with your university's own estimate" and "Indicative. Confirm with your AD bank."; the engine only adds up and converts (taka rounded up, so a month is never under-stated); the arrival hints say to confirm visa, work and registration rules with the university and the official page; the assistant repeats engine figures only (number guard) and never says something is legal	Accepted: real data needs a real source
R44	**"Create draft and request approval" changes two states with one click.** A person could request approval without reading the draft	Low	It only moves the payment to PENDING_APPROVAL; nothing is approved, submitted or sent. Approving needs the approval summary, its hash and a fresh MFA proof, as before, and the parent can reject or cancel. Both calls carry Idempotency-Keys and are audited; the button says exactly what it does	Accepted
R45	**Chat text leaves the server for a third party** when a person uses their own Gemini or Groq key, or an operator turns on the Hugging Face Space	Medium	Off by default: with nothing configured the assistant runs offline in template mode. Using a key is the person's choice, and saving it needs explicit consent. Names, accounts, passports, e-mails and phones are masked before any text leaves; the provider hosts are fixed in code (no user-supplied URL, no redirects, HTTPS only); replies are size-limited and still pass the number guard and the tool guard; keys and provider bodies never appear in logs or errors	Accepted: what a provider keeps is outside our control. The AI can still only propose
R46	**A saved provider API key is a secret kept in the database** (one per signed-in person)	Medium	AES-256-GCM through the project's own crypto helper, bound to the person and the provider, so a ciphertext cannot be moved to another row; the key is never returned (the screen gets only the provider, model and a verified flag); removal and the privacy reset delete it; saving needs consent and a synthetic test call first	Accepted: someone who holds both the database and `DATA_ENCRYPTION_KEY` can read it (the sandbox keeps dev secrets under `data/`, R13). Use a limited key
R47	**Cookie sessions bring cross-site request and session risks**, and script can still act as the signed-in person if XSS ever happens	Medium	One opaque `sr_browser_session` cookie: HttpOnly, SameSite=Strict, limited to `/api/v1`, Secure outside the plain-HTTP sandbox, stored only as a hash, absolute expiry, revoked at sign-out and replaced at the next sign-in. Browser sign-in needs an exact trusted Origin and a custom header; an access token is bound to its session and refused without the cookie; the cookie alone is never API authority. Probe checks 6.10 to 6.21 and the 42-check browser audit test all of this	Accepted: same-origin use only (opening the app by an address that is not in `CORS_ORIGINS` is refused on purpose); XSS defence stays with the CSP
R48	**The operator's Hugging Face Space** is called with a server-side token and a shared secret, and has not been deployed or tested live	Low	Used only when `ASSISTANT_CLOUD_ENABLED=true`; the URL must be one exact `https://name.hf.space` origin; the token and secret live only in the server's environment; no model runs locally and no extra library is needed; failures are shown as typed messages (not connected, quota, busy) and never fall back silently	Open until the Space is deployed and a real call is recorded

ID	Risk	Severity	Mitigation in place	Residual status
R49	**Uploaded document text was stored in plain text** in the database file, so anyone who could copy the file could read pasted passport numbers (found by `leak_scan.py`, 17 Oct 2026)	Medium	Fixed: the text is AES-256-GCM encrypted with the project's field cipher, bound to its own row, with a fresh nonce each time; older plain rows are still readable. File names and recipient names stay plain on purpose (they are shown in lists). `test_documents_at_rest.py`; the leak scan now fails if the text appears in the database	Fixed. The sandbox key sits next to the database (R13): a real deployment keeps it in a key vault
R50	**9 high-severity advisories in development tools** (`shadcn` CLI, `eslint-config-next`, through `braces`, `micromatch`, `fast-glob`)	Low	Build-time only: none is in the production dependency tree, `npm audit --omit=dev` finds 0 and CI runs exactly that. The advisory is a pattern-matching denial of service, reachable only by feeding the tool hostile glob patterns	Accepted: the fix is a major upgrade of a build tool; revisit after submission
R51	**The bank review could be read as a legal or anti-money-laundering verdict, or as profiling a family.** A flagged payment might be mistaken for an accusation, and a large or split payment is usually innocent	Medium	The AI reading and the plain-rule context are figures and reasons, never a conclusion; the wording is fixed in one file and a test fails if any page or assistant answer uses words such as illegal, fraud or laundering; every page says "Indicative. Confirm with your AD bank."; a person decides; the parent is always told in plain words why and what to prepare; thresholds are config marked `verified: false`. `docs/BANK_REVIEW.md`	Accepted: the sandbox is not a screening system. A real bank brings its own lists, thresholds and people
R52	**The reviewer role can now issue MFA proofs and decide on any household's payment**	Medium	A decision needs a fresh single-use MFA proof for that payment under its own scope (`review_payment`), so a proof made to approve a payment is refused; the reviewer cannot approve, cancel or change a limit; a yes does not replace the parent's own approval; the AI and the parent cannot decide (tested); every step is audited with the actor	Accepted: the demo has one reviewer account. A real bank would use named reviewers and a second pair of eyes for large amounts
R53	**The risk model learned from synthetic data and decides who a person looks at, so it could route unfairly or wrongly.** It might send an innocent family's payment to a person, or let an odd one take the fast lane	Medium	It only routes: it cannot clear, decline or approve (the AI has no review permission, tested), the parent still approves with MFA, a person with a fresh MFA proof answers, and the sandbox caps and the name-list match still apply to a payment it rates low; it sees no name, nationality or account number (size next to other payments, how often this household paid, whether the receiver is new); the parent and the reviewer are shown its band, how that band fared in practice data and the factors that drove it, so a wrong reading can be argued with; its name, band and routing are written to the audit log; the files are checked against a SHA-256 manifest and `RISK_MODE=off` returns to plain rules (the page says "No AI reading"); trained on a public data set under CDLA-Sharing-1.0 and judged on a later period it never saw. Its limits are published, including that precision and recall cannot both pass 90% on this data (`docs/AI_RISK.md`)	Accepted: not validated on real payments or on this corridor; wire and cross-currency payments carry no label in the data
R54	**The small explainer model could write something untrue or unsafe in a reply**	Medium	It can only rewrite the engine's own draft. The rewrite is thrown away, and the engine's draft shown, if it adds a figure, drops most figures, loses the disclaimer or a refusal, names a provider without "(synthetic)", claims an action or calls anything legal; the existing number guard and tool guard still run; the model files are ONNX data checked against a SHA-256 manifest; it runs locally with no network; `EXPLAINER_MODE=off` removes it. `docs/AI_EXPLAINER.md`	Accepted: wording can be clumsy; trained on synthetic questions and judged on synthetic ones

ID	Risk	Severity	Mitigation in place	Residual status
R55	**A parent's answer to the bank can contain personal details**	Low	Stored sealed with AES-256-GCM bound to its own row (tested: a canary string is not in the database file), shown only to that household and the reviewer, limited to 500 characters, the audit log keeps only its hash and length, the screen tells the parent not to type passwords or account numbers, and "Reset my data" deletes it	Accepted: a real system would also set a retention period

3. What we tested and how to repeat it

```
cd backend && pytest -q               # blueprint T01-T24 plus AI, sandbox and demo tests
python -m app.scripts.demo_flow       # fourteen-step demo against a running API
python -m app.scripts.tamper_audit    # edit one audit row, then Verify must FAIL at that sequence
python scripts/scan_repo.py           # no secrets, real e-mails, passport-like numbers or TODOs
python ml/evaluate.py                 # the trained reader against the rule-based baseline (docs/AI_EVALUATION.md)
```

Operation logs from a real run are in `submission/evidence/`.